Project Title	**Air Tracker: Flight Analytics**
Skills take away From This Project	**Python scripting, Data Collection using API integration, Data Management using SQL, Streamlit**
Domain	**Aviation/Data Analytics**

Problem Statement:

The AeroDataBox Flight Explorer project aims to develop a comprehensive solution for managing, visualizing, and analyzing aviation data extracted from the AeroDataBox API. The application will parse JSON data, store structured information in a relational database, and provide intuitive insights into airports, flight schedules, and operational details. This project is designed to assist aviation enthusiasts, analysts, and organizations in understanding airport networks, flight patterns, and real-time operations while exploring detailed flight information interactively.

Business Use Cases:

1. Airport Exploration: Enable users to navigate airport details including locations and timezones.
2. Flight Trend Analysis: Visualize distributions of departures and arrivals by status, airline, or time.
3. Operational Insights: Analyze flight delays and statuses across selected airports.
4. Decision Support: Offer data-driven insights to airport managers or travel planners for scheduling and resource allocation.

Approach:

Data Extraction

- Parse and extract data from AeroDataBox JSON responses using the API.
- Transform nested JSON structures into a flat relational schema for analysis.

Data Storage:

- Create a SQL database with well-designed schema (e.g., defining appropriate data types and primary keys). The data to be collected is provided below.

Data Collection:

1) STEPS TO COLLECT THE DATA FROM THE API ENDPOINTS-

```
Sample code (for example to get airport information) :
API_HOST = "aerodatabox.p.rapidapi.com"
API_KEY = "your_api_key"

HEADERS = {
    'x-rapidapi-key': API_KEY,
    'x-rapidapi-host': API_HOST
}
   url = f"https://{API_HOST}/airports/iata/{iata_code}"
    response = requests.get(url, headers=HEADERS)
    response.raise_for_status()
    return response.json()
```

Similarly you can explore other endpoints provided for different kinds of information on this page:

AeroDataBox rapidAPI page

GetFlight API documentation

Note: Select 10-15 airport codes of your choice and then fetch data for these airports using Airports API endpoints, Then fetch only the flights that either arrived at or departed from these airports in the past few days. After getting the flights info, you can check unique aircraft codes in the flights table and fetch the detailed info of these aircrafts using the aircrafts API endpoints. To get the airport delays data, use statistical API endpoints.

Table Structure

airport

- airport_id INTEGER PRIMARY KEY AUTOINCREMENT,
- icao_code TEXT UNIQUE,
- iata_code TEXT UNIQUE,
- name TEXT,
- city TEXT,
- country TEXT,
- continent TEXT,
- latitude REAL,
- longitude REAL,
- timezone TEXT

aircraft

- aircraft_id INTEGER PRIMARY KEY AUTOINCREMENT,
- registration TEXT UNIQUE,
- model TEXT,
- manufacturer TEXT,
- icao_type_code TEXT,
- owner TEXT

flights

- flight_id TEXT PRIMARY KEY,
- flight_number TEXT,
- aircraft_registration TEXT,
- origin_iata TEXT,
- destination_iata TEXT,
- scheduled_departure TEXT,
- actual_departure TEXT,
- scheduled_arrival TEXT,
- actual_arrival TEXT,
- status TEXT,
- airline_code TEXT
- Additional fields as required...

airport_delays

- delay_id INTEGER PRIMARY KEY AUTOINCREMENT,
- airport_iata TEXT,
- delay_date TEXT,
- total_flights INTEGER,

- delayed_flights INTEGER,
- avg_delay_min INTEGER,
- median_delay_min INTEGER,
- canceled_flights INTEGER

Execute the following SQL queries:

1) Show the total number of flights for each aircraft model, listing the model and its count.
2) List all aircraft (registration, model) that have been assigned to more than 5 flights.
3) For each airport, display its name and the number of outbound flights, but only for airports with more than 5 flights.
4) Find the top 3 destination airports (name, city) by number of arriving flights, sorted by count descending.
5) Show for each flight: number, origin, destination, and a label 'Domestic' or 'International' using CASE WHEN on country match.
6) Show the 5 most recent arrivals at “DEL” airport including flight number, aircraft, departure airport name, and arrival time, ordered by latest arrival.
7) Find all airports with no arriving flights (never used as a destination in flights table).
8) For each airline, count the number of flights by status (e.g., 'On Time', 'Delayed', 'Cancelled') using CASE WHEN.
9) Show all cancelled flights, with aircraft and both airports, ordered by departure time descending.
10) List all city pairs (origin-destination) that have more than 2 different aircraft models operating flights between them.
11) For each destination airport, compute the % of delayed flights (status='Delayed') among all arrivals, sorted by highest percentage.

Build a Streamlit Application:

- Connect the Streamlit app with the SQL database for real-time query execution.
- Display analysis results in the form of tables, charts, or dashboards within the app.**User Interface**: Interactive dashboards with filters for competition types, levels, and categories.

Suggested Features for the Application (Use your creativity)

1. **Homepage Dashboard:**
    a. Summary statistics like:
        i. Total number of airports.
        ii. Total flights fetched.
        iii. Average delay across airports.
2. **Search and Filter Flights:**
    a. Allow users to search for flights by number or airline.
    b. Filter flights by status, origin, or date range.
3. **Airport Details Viewer:**
    a. Display detailed information about a selected airport, including:
        i. Location, timezone, and linked flights.
4. **Delay Analysis:**
    a. Visualize delay percentages and averages by airport.
5. **Route Leaderboards:**
    a. Show tables for:
        i. Busiest routes (most flights).
        ii. Most delayed airports.

Results

The expected outcome is a fully functional web application capable of parsing and visualizing aviation data. Users can navigate airport networks, filter flights, and generate meaningful insights about operations and delays. This project will enhance proficiency in working with APIs, databases, and visualization tools.

Skills Takeaway

1. Data Extraction: API calls and JSON parsing.
2. SQL Database Management: Designing tables and schema.
3. Data Analysis: Writing SQL queries to derive insights.
4. Visualization: Building interactive Streamlit dashboards.

Project Evaluation Metrics

1. Data Extraction Accuracy: Successful retrieval of data from API without errors or missing values.
2. SQL Database Design: Proper structuring of tables with normalized data and correct relationships.

3. Query Efficiency: Ability to write optimized SQL queries that retrieve relevant insights quickly.
4. Streamlit Application Functionality: Smooth navigation and interactive features for data display.
5. Project Completeness: End-to-end workflow from data extraction to SQL storage and Streamlit app.
6. Documentation Quality: Clearly upload all files in GitHub.
7. Presentation and Usability: Intuitive user interface with meaningful outputs and insights.
8. Error Handling: Manage API rate limits, database constraints, or input errors effectively.
9. Innovation and Creativity: Unique ideas or insightful SQL queries.

Technical Tags

1. Languages: Python
2. Database: MySQL/PostgreSQL
3. Application: Streamlit
4. API Integration: AeroDataBox API

Project Deliverables

1. SQL Database: Populated with structured aviation data from the AeroDataBox API.
2. API Scripts: Automate data extraction and transform JSON into relational format.
3. Streamlit App: Interactive tool for exploring and visualizing flight and airport data.
4. Documentation: Detailed report on workflow, schema design, challenges, and insights.

Project Guidelines

1. Coding Standards
    - Use meaningful names for variables, functions, and tables.
    - Follow PEP 8 for Python: Consistent formatting with indentation.
    - Modularize code: Break into functions for readability.
    - Error handling: Use try-except for API and SQL errors.
    - Document code: Include docstrings and comments.
2. SQL Database Practices
    - Normalize tables: Avoid redundancy.

- Use indexes: Optimize query performance.
- Consistent naming: Descriptive for tables and fields.

3. Streamlit Application Development
    - Interactive features: Responsive UI with widgets for filters.
    - Minimalist design: Simple layout for user experience.
    - Performance: Use pagination or batch loading.
4. General Best Practices
    - Test frequently: Each component during development.
    - Backup data: Database and code.
    - Documentation: README with setup, objectives, and demo.

** ***In github, upload all your code files. Also upload a document that contains all the SQL queries.***

Reference:

If you don't know how to approach the project, kindly refer to the project orientation recording provided in this table. (Available in English & Hindi)

Streamlit Doc	https://docs.streamlit.io/library/api-reference
SAMPLE PROJECT FOR REFERENCE(ENGLISH)	Project Excellence Series: Guided Learning …
Streamlit recording (English)	Special session for STREAMLIT(11/08/2…
Project Live Evaluation Metrics	Project Live Evaluation
Capstone Explanation Guideline	Capstone Explanation Guideline

GitHub Reference	How to Use GitHub.pptx
sample code to get airport data	Airport_data_demo.ipynb (change the API key and airport codes according to you, do not use exact same code, this is just for reference)
Project Orientation (English)	Project Orientation Session_DS-C-WE-…
Project Orientation (Hindi)	Project orientation session Hindi Project Orientation Session : Air Tracker…

Timeline:

The project must be completed and submitted within **14 days from the assigned date.**

PROJECT DOUBT CLARIFICATION SESSION (PROJECT AND CLASS DOUBTS)

About Session: The Project Doubt Clarification Session is a helpful resource for resolving questions and concerns about projects and class topics. It provides support in understanding project requirements, addressing code issues, and clarifying class concepts. The session aims to enhance comprehension and provide guidance to overcome challenges effectively.

Timing: Monday to Saturday (3:30 PM to 4:30 PM)

Session link and PDS guidelines: Project Doubt Session [DS/AIML]

LIVE EVALUATION SESSION (CAPSTONE AND FINAL PROJECT)

About Session: The Live Evaluation Session for Capstone and Final Projects allows participants to showcase their projects and receive real-time feedback for improvement. It assesses project quality and provides an opportunity for discussion and evaluation.
Note: This form will Open on Saturday and Sunday Only on Every Week

Timing: Monday-Saturday (5:30 PM to 6:30 PM)

Booking link : https://forms.gle/1m2Gsro41fLtZurRA

Project Created By	Verified By	Approved By
Kirti Gupta	Shadiya P P	Nehlath Harmain